

✓ Explainable Cyber Threat Intelligence (CTI) Agent — trained with GRPO

```
# Goal: teach Qwen2.5-3B-Instruct to read a CTI text snippet and output the
# correct MITRE ATT&CK Technique ID inside <answer> tags, preceded by
# step-by-step reasoning inside <reasoning> tags.
#
# Algorithm: GRPO (Group Relative Policy Optimization) via TRL.
#
# Why GRPO instead of PPO?
# -----
# PPO needs a separate Value/Critic model (typically as large as the policy)
# to estimate the expected reward of every token. That doubles GPU memory and
# struggles with credit assignment on long reasoning chains.
#
# GRPO removes the Critic entirely. For each prompt it samples a *group* of G
# completions, scores each one with cheap, deterministic reward functions
# (rule-based Python – no neural network), and computes each completion's
# advantage RELATIVE to the group:
#
#           A_i = (r_i - mean(r)) / std(r)
#
# Completions that beat the group average are reinforced; those below it are
# pushed down. No Critic => ~half the memory => fits a single 16GB Kaggle GPU.
#
# Target hardware: Kaggle T4 (compute 7.5, fp16 – NO bf16).
# =====
```

✓ Installs (run these in the first Kaggle cell, then RESTART the session)

```
# Unsloth pins a compatible trl + vllm for you; version drift between
# unsloth / trl / vllm / torch is the #1 cause of import errors on Kaggle.
# Kaggle already ships a working torch build, so do NOT reinstall torch.
#
!pip install -q unsloth vllm
!pip install -q --no-deps "trl==0.15.2" datasets

#
# If you hit a vLLM/torch mismatch, prefer the single line:
!pip install -q "unsloth[kaggle-new]" vllm
# This installs the libraries quietly, hiding the Kaggle dependency warnings
!pip install -q transformers peft trl datasets bitsandbytes accelerate > /dev/null 2>&1
# =====
```

```
15/32215 MB 112 MB/s eta 0:00:00
.3/998.3 kB 69.8 MB/s eta 0:00:00
7/60.7 MB 19.6 MB/s eta 0:00:00
.8/506.8 kB 37.0 MB/s eta 0:00:00
.5/118.5 kB 13.7 MB/s eta 0:00:00
/3.0 MB 118.8 MB/s eta 0:00:00
/6.6 MB 128.9 MB/s eta 0:00:00
.7/125.7 kB 13.7 MB/s eta 0:00:00
/2.2 MB 99.2 MB/s eta 0:00:00
/2.8 MB 106.8 MB/s eta 0:00:00
5/88.5 MB 9.3 MB/s eta 0:00:00
```

```
.6/222.6 kB 22.4 MB/s eta 0:00:00
.0/225.0 kB 24.4 MB/s eta 0:00:00
.7/180.7 kB 19.7 MB/s eta 0:00:00
6/71.6 kB 7.4 MB/s eta 0:00:00
.0/215.0 kB 24.0 MB/s eta 0:00:00
/2.7 MB 108.3 MB/s eta 0:00:00
.4/819.4 kB 50.0 MB/s eta 0:00:00
4/42.4 MB 19.5 MB/s eta 0:00:00
1/50.1 MB 13.9 MB/s eta 0:00:00
5/79.5 kB 8.2 MB/s eta 0:00:00
7/61.7 kB 5.2 MB/s eta 0:00:00
.7/320.7 kB 31.1 MB/s eta 0:00:00
/3.2 MB 100.3 MB/s eta 0:00:00
6/61.6 kB 6.4 MB/s eta 0:00:00
.8/897.8 kB 63.7 MB/s eta 0:00:00
.1/331.1 kB 31.1 MB/s eta 0:00:00
7/77.7 kB 8.4 MB/s eta 0:00:00
/4.8 MB 117.2 MB/s eta 0:00:00
/8.0 MB 109.7 MB/s eta 0:00:00
.8/959.8 kB 70.3 MB/s eta 0:00:00
```

ntly take into account all the packages that are installed. This behaviour is the source of the following dependency t installed.

2.0, but you have huggingface-hub 0.36.2 which is incompatible.

=5.26.1, but you have protobuf 6.33.6 which is incompatible.

42.1,>=1.39, but you have opentelemetry-api 1.44.0 which is incompatible.

42.1,>=1.39, but you have opentelemetry-sdk 1.44.0 which is incompatible.

otobuf!=4.21.0,!<4.21.1,!<4.21.2,!<4.21.3,!<4.21.4,!<4.21.5,<6.0.0dev,>=3.20.2, but you have protobuf 6.33.6 which i

```
import os
```

```
# IMPORTANT: must be set BEFORE `import unsloth`.
```

```
# On Kaggle T4 (compute 7.5), vLLM tries to JIT-compile FlashInfer's sampling
```

```
# kernel and the link step fails with "/usr/bin/ld: cannot find -lcuda" (the
```

```
# CUDA driver stub isn't on the linker path in the Kaggle image). Installing
```

```
# ninja does NOT fix it – ninja runs fine; the linker is what fails. Disabling
```

```
# FlashInfer makes vLLM fall back to its native Triton top-k/top-p sampler,
```

```
# which needs no compilation and works on the T4.
```

```
os.environ["UNSLOTH_VLLM_NO_FLASHINFER"] = "1"
```

```
import ast
```

```
import re
```

```
import torch
```

```
from datasets import Dataset, load_dataset
```

```
from unsloth import FastLanguageModel
```

```
from trl import GRPOConfig, GRPOTrainer
```

```
# -----
# 1. Model initialization – Unsloth FastLanguageModel + vLLM fast inference
# -----
```

```
# NOTE: we load the 7B via Unsloth's FastLanguageModel (NOT plain
```

```
# AutoModelForCausalLM) because the whole GRPO pipeline needs 4-bit weights +
```

```
# vLLM fast generation + LoRA training, which the vanilla transformers path
```

```
# from the model card cannot provide.
```

```
#
```

```
# HEADS-UP on hardware: Qwen2-7B in 4-bit + vLLM KV-cache + LoRA training +
```

```
# sampling G completions is TIGHT on a single 16GB T4. It generally fits with
```

```
# the reduced settings below, but if you OOM, in order: (a) drop num_generations
```

```
# AND per_device_train_batch_size to 2 together, (b) lower max_completion_length
```

```
# to 200, (c) lower gpu_memory_utilization. On P100 (no bf16, weaker vLLM) prefer
```

```
# the 3B model instead.
```

```
MAX_SEQ_LENGTH = 1024 # prompt (CTI snippet) + completion budget
```

```
LORA_RANK = 16 # smaller rank than the 3B run to save 7B memory
```

```
# (must stay <= max_lora_rank below)
```

```
model, tokenizer = FastLanguageModel.from_pretrained(
```

```
    model_name="Qwen/Qwen2.5-1.5B-Instruct",
```

```
    max_seq_length=MAX_SEQ_LENGTH,
```

```
    load_in_4bit=True, # 4-bit QLoRA base – mandatory to fit 7B on a T4
```

```
    fast_inference=True, # route generation through vLLM.
```

```
    # GRPO's bottleneck is sampling G completions
```

```
    # per prompt; vLLM makes that ~10x faster.
```

```
    max_lora_rank=LORA_RANK,
```

```
    gpu_memory_utilization=0.55, # vLLM KV-cache share. Kept modest so the 7B
```

```
    # LoRA forward/backward still fits in 16GB.
```

```
    dtype=torch.float16, # T4 has NO bf16 – force fp16 everywhere.
```

```
)
```

```
# Attach LoRA adapters. We only train these: the 4-bit base stays frozen.
```

```

model = FastLanguageModel.get_peft_model(
    model,
    r=LORA_RANK,
    target_modules=[
        "q_proj", "k_proj", "v_proj", "o_proj",  # attention projections
        "gate_proj", "up_proj", "down_proj",    # MLP projections
    ],
    lora_alpha=LORA_RANK,
    use_gradient_checkpointing="unsloth",  # Unsloth's memory-efficient variant
    random_state=3407,
)

# -----
# 2. Dataset – tumetector/Security-TTP-Mapping (CTI text -> MITRE technique IDs)
# -----
# Schema: column "text1" (CTI snippet) and "labels" (a *stringified* Python list
# such as ["T1057"]). Rows may carry multiple technique IDs, and IDs may be
# sub-techniques (e.g. "T1059.001").
#
# The system prompt below forces the exact output contract we reward for.
SYSTEM_PROMPT = (
    "You are a cyber threat intelligence analyst. You are given a threat "
    "intelligence text snippet describing adversary behavior. Identify the "
    "single most relevant MITRE ATT&CK technique.\n"
    "Think step-by-step inside <reasoning> tags and output the exact MITRE "
    "ATT&CK ID (e.g. T1059 or T1059.001) inside <answer> tags.\n"
    "Respond in EXACTLY this format and nothing else:\n"
    "<reasoning>\n...your step-by-step analysis...\n</reasoning>\n"
    "<answer>T####</answer>"
)

def _parse_labels(raw):
    """Turn a stringified label list into a clean list of technique IDs."""
    if isinstance(raw, list):
        ids = raw
    else:
        try:
            ids = ast.literal_eval(str(raw))
        except (ValueError, SyntaxError):
            # sometimes labels arrive as a bare string like "T1057"
            ids = [raw]
        if not isinstance(ids, list):
            ids = [ids]
    # normalize: uppercase, strip, drop empties
    return [str(i).strip().upper() for i in ids if str(i).strip()]

def _to_grpo_row(example):
    """Map a raw dataset row into the conversational format GRPOTrainer wants."""
    ids = _parse_labels(example["labels"])
    return {
        "prompt": [
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": f"CTI snippet:\n{example['text1']}"},
        ],
        # extra columns are forwarded verbatim to the reward functions as kwargs
        "ground_truth": ids,
    }

def build_dataset():
    """Load Security-TTP-Mapping; fall back to a tiny synthetic set if offline."""
    try:
        raw = load_dataset("tumetector/Security-TTP-Mapping", split="train")
        ds = raw.map(_to_grpo_row, remove_columns=raw.column_names)
        # drop rows whose labels failed to parse into at least one ID
        ds = ds.filter(lambda r: len(r["ground_truth"]) > 0)
        print(f"[data] loaded Security-TTP-Mapping: {len(ds)} rows")
        return ds
    except Exception as e:  # noqa: BLE001 – Kaggle may block the download
        print(f"[data] load failed ({e!r}); using synthetic fallback set")
        samples = [
            ("Process 'cmd.exe' spawned 'powershell.exe -enc JABzAD0A...'", "T1059.001"),
            ("The malware queried the registry to enumerate running processes.", "T1057"),
            ("A scheduled task was created to run the payload every 10 minutes.", "T1053.005"),
            ("Credentials were dumped from LSASS memory using a custom tool.", "T1003.001"),
        ]

```

```

        ("The sample established a TLS connection to 198.51.100.34:443 for C2.", "T1071.001"),
        ("Files in the user directory were encrypted and a ransom note dropped.", "T1486"),
        ("The binary copied itself to the Startup folder for persistence.", "T1547.001"),
        ("It executed 'whoami' and 'ipconfig /all' to profile the host.", "T1082"),
    ]
    rows = [
        _to_grpo_row({"text1": text, "labels": f"['{tid}']"})
        for text, tid in samples
    ]
    return Dataset.from_list(rows)

dataset = build_dataset()

# -----
# 3. Reward functions
# -----
# GRPOTrainer calls each reward function with (prompts, completions, **kwargs).
# `completions` is a list (one per sampled generation) of message lists in
# conversational format, so the assistant text is completions[i][0]["content"].
# Any extra dataset column – here `ground_truth` – is forwarded as a same-length
# list keyword argument. Each function returns a list[float] of rewards.

# Strict full-output contract: <reasoning>...</reasoning><answer>...</answer>
_STRICT_FORMAT = re.compile(
    r"^\s*<reasoning>.*?</reasoning>\s*<answer>.*?</answer>\s*$",
    re.DOTALL,
)

# Pull the text inside the FIRST <answer>...</answer>
_ANSWER = re.compile(r"<answer>\s*(.*?)\s*</answer>", re.DOTALL)
# A MITRE technique ID, optionally with a sub-technique (T1059 / T1059.001)
_TID = re.compile(r"T\d{4}(?:\.\d{3})?", re.IGNORECASE)

def _completion_text(completion):
    """Extract the assistant string from one conversational completion."""
    return completion[0]["content"]

def format_reward_func(prompts, completions, **kwargs):
    """1.0 if the completion strictly matches the required XML format, else 0.0."""
    rewards = []
    for c in completions:
        text = _completion_text(c)
        rewards.append(1.0 if _STRICT_FORMAT.match(text) else 0.0)
    return rewards

def _extract_id(text):
    """Return the normalized technique ID inside <answer>, or None."""
    m = _ANSWER.search(text)
    if not m:
        return None
    hit = _TID.search(m.group(1))
    return hit.group(0).upper() if hit else None

def correctness_reward_func(prompts, completions, ground_truth=None, **kwargs):
    """+2.0 if the predicted ID exactly matches ANY ground-truth ID, else -1.0.

    Multi-label aware: a row may list several valid techniques. Exact match is
    required, so a sub-technique like T1059.001 does NOT satisfy a bare T1059
    (and vice versa) unless that exact ID is in the label list.
    """
    if ground_truth is None:
        ground_truth = [[] for _ in completions]
    rewards = []
    for c, truth in zip(completions, ground_truth):
        pred = _extract_id(_completion_text(c))
        truth_set = {t.strip().upper() for t in truth}
        rewards.append(2.0 if pred is not None and pred in truth_set else -1.0)
    return rewards

# Optional soft nudge: partial credit for producing the tags at all. This eases
# the cold-start phase before the model reliably emits the full format. Tunable –
# remove it from `reward_funcs` below if you want a purely strict signal.
def soft_format_reward_func(prompts, completions, **kwargs):

```

```

    rewards = []
    for c in completions:
        text = _completion_text(c)
        score = 0.0
        if "<reasoning>" in text and "</reasoning>" in text:
            score += 0.25
        if "<answer>" in text and "</answer>" in text:
            score += 0.25
        rewards.append(score)
    return rewards

# -----
# 4. GRPO configuration
# -----
# num_generations (G) is the group size scored per prompt. G=4 is the practical
# ceiling for a 7B model in 4-bit on a 16GB T4 (G=8 will OOM during generation);
# drop to G=2 if you still run out of memory.
#
# IMPORTANT divisibility rule (TRL will assert this):
# (per_device_train_batch_size * gradient_accumulation_steps * num_gpus)
# must be divisible by num_generations.
# Here 4 * 1 * 1 = 4, and 4 % 4 == 0. If you hit OOM, drop BOTH num_generations
# and per_device_train_batch_size together (e.g. to 2) so they stay divisible.
NUM_GENERATIONS = 4

training_args = GRPOConfig(
    output_dir="/kaggle/working/outputs",
    # --- GRPO group sampling ---
    num_generations=NUM_GENERATIONS,
    max_prompt_length=640,
    max_completion_length=256,
    use_vllm=True,          # generate with the vLLM engine loaded above
    # --- batching (see divisibility note) ---
    per_device_train_batch_size=NUM_GENERATIONS,
    gradient_accumulation_steps=1,
    # --- precision: T4 => fp16 only, bf16 unavailable ---
    fp16=True,
    bf16=False,
    # --- optimization ---
    learning_rate=5e-6,      # RL fine-tuning likes a small LR
    optim="adamw_8bit",      # 8-bit optimizer to save memory
    warmup_ratio=0.1,
    lr_scheduler_type="cosine",
    weight_decay=0.1,
    max_grad_norm=0.1,
    # --- logging / duration / saving ---
    logging_steps=1,
    max_steps=250,           # demo length; raise (or use num_train_epochs)
                             # for a real run once rewards look healthy.
    save_steps=250,
    report_to="none",
)

# -----
# 5. Trainer
# -----
trainer = GRPOTrainer(
    model=model,
    processing_class=tokenizer,
    reward_funcs=[
        format_reward_func,      # 0.0 / 1.0 - strict XML contract
        correctness_reward_func, # -1.0 / +2.0 - right MITRE ID (dominant signal)
        soft_format_reward_func, # 0.0..0.5 - cold-start scaffolding (optional)
    ],
    args=training_args,
    train_dataset=dataset,
)

# -----
# 6. Train + save the LoRA adapters
# -----
if __name__ == "__main__":
    trainer.train()

    # Save just the trained LoRA adapters to the Kaggle working dir (downloadable).
    model.save_lora("/kaggle/working/grpo_cti_lora")

```

```
model.save_pretrained("/kaggle/working/grpo_cti_adapters")
tokenizer.save_pretrained("/kaggle/working/grpo_cti_adapters")
print("[done] LoRA adapters saved to /kaggle/working/")

# To ship a standalone 16-bit model instead of adapters, use:
# model.save_pretrained_merged(
#     "/kaggle/working/grpo_cti_merged", tokenizer, save_method="merged_16bit")

# -----
# Reward smoke test (run BEFORE the expensive train() to sanity-check scoring).
# Copy this block into a scratch cell – it needs no GPU.
# -----
def _msg(text):
    return [{"role": "assistant", "content": text}]
#
# good = _msg("<reasoning>cmd spawns encoded powershell => command interpreter"
#             "</reasoning>\n<answer>T1059.001</answer>")
# bad_fmt = _msg("The answer is T1059.001")
# wrong = _msg("<reasoning>looks like discovery</reasoning>\n<answer>T1082</answer>")
# gt = [{"T1059.001"}]
#
# assert format_reward_func(None, [good[:1]]) == [1.0]
# assert format_reward_func(None, [bad_fmt[:1]]) == [0.0]
# assert correctness_reward_func(None, [good[:1]], ground_truth=gt) == [2.0]
# assert correctness_reward_func(None, [wrong[:1]], ground_truth=gt) == [-1.0]
# print("reward functions OK")
```



```

🐼 Unsloth: Will patch your computer to enable 2x faster free finetuning.
🐼 Unsloth Zoo will now patch everything to make training faster!
INFO 07-17 12:25:11 [vllm_utils.py:739] Unsloth: Patching vLLM v1 graph capture
==(=====) Unsloth 2026.7.3: Fast Qwen2 patching. Transformers: 4.57.6. vLLM: 0.19.1.
    \ \ / /
    / / \ \ Tesla T4. Num GPUs = 1. Max memory: 14.563 GB. Platform: Linux.
0~0/ \ \ / \ Torch: 2.10.0+cu128. CUDA: 7.5. CUDA Toolkit: 12.8. Triton: 3.6.0
 \ \ / / Bfloat16 = FALSE. FA [Xformers = 0.0.35. FA2 = False]
"-__-" Free license: http://github.com/unslothai/unsloth
Unsloth: Fast downloading is enabled - ignore downloading bars which are red colored!
Unsloth: vLLM loading unsloth/qwen2.5-1.5b-instruct-unsloth-bnb-4bit with actual GPU utilization = 54.36%
Unsloth: Your GPU has CUDA compute capability 7.5 with VRAM = 14.56 GB.
Unsloth: Using conservativeness = 1.0. Chunked prefill tokens = 1024. Num Sequences = 32.
Unsloth: vLLM's KV Cache can use up to 6.53 GB. Also swap space = 0 GB.
Unsloth: Patched vLLM's piecewise graph-partition pass so compilation_config=3 (CUDA graphs) works with LoRA.
Unsloth: Not an error, but `level` is not supported in vLLM.config.CompilationConfig. Skipping.
Unsloth: Not an error, but `use_cudagraph` is not supported in vLLM.config.CompilationConfig. Skipping.
Unsloth: Not an error, but `use_inductor` is not supported in vLLM.config.CompilationConfig. Skipping.
Unsloth: Not an error, but `swap_space` is not supported in vLLM. Skipping.
Unsloth: Not an error, but `device` is not supported in vLLM. Skipping.
INFO 07-17 12:25:18 [utils.py:233] non-default args: {'load_format': 'bitsandbytes', 'dtype': torch.float16, 'max_mod
<frozen importlib_bootstrap_external>:1301: FutureWarning: The cuda.cudart module is deprecated and will be removed
<frozen importlib_bootstrap_external>:1301: FutureWarning: The cuda.nvrtc module is deprecated and will be removed i
WARNING 07-17 12:25:19 [arg_utils.py:1390] The global random seed is set to 0. Since VLLM_ENABLE_V1_MULTIPROCESSING i
INFO 07-17 12:25:40 [model.py:549] Resolved architecture: Qwen2ForCausalLM
WARNING 07-17 12:25:40 [model.py:2016] Casting torch.bfloat16 to torch.float16.
INFO 07-17 12:25:40 [model.py:1678] Using max model len 1024
INFO 07-17 12:25:40 [scheduler.py:238] Chunked prefill is enabled with max_num_batched_tokens=4096.
Unsloth: vLLM Bitsandbytes config using kwargs = {'load_in_8bit': False, 'load_in_4bit': True, 'bnb_4bit_compute_dtyp
INFO 07-17 12:25:40 [vllm.py:790] Asynchronous scheduling is enabled.

tokenizer_config.json:      7.36k/? [00:00<00:00, 472kB/s]

vocab.json:      2.78M/? [00:00<00:00, 61.1MB/s]

merges.txt:      1.67M/? [00:00<00:00, 49.3MB/s]

tokenizer.json: 100%      11.4M/11.4M [00:00<00:00, 22.3MB/s]

added_tokens.json: 100%      605/605 [00:00<00:00, 42.6kB/s]

special_tokens_map.json: 100%      614/614 [00:00<00:00, 53.3kB/s]

generation_config.json: 100%      270/270 [00:00<00:00, 21.6kB/s]
INFO 07-17 12:25:44 [core.py:105] Initializing a V1 LLM engine (v0.19.1) with config: model='unsloth/qwen2.5-1.5b-ins
INFO 07-17 12:25:44 [parallel_state.py:1400] world_size=1 rank=0 local_rank=0 distributed_init_method=tcp://172.28.0.
INFO 07-17 12:25:44 [parallel_state.py:1716] rank 0 in world size 1 is assigned as DP rank 0, PP rank 0, PCP rank 0,
INFO 07-17 12:25:45 [gpu_model_runner.py:4735] Starting to load model unsloth/qwen2.5-1.5b-instruct-unsloth-bnb-4bit.
ERROR 07-17 12:25:46 [fa_utils.py:145] Cannot use FA version 2 is not supported due to FA2 is only supported on devic
INFO 07-17 12:25:46 [cuda.py:334] Using FLASHINFER attention backend out of potential backends: ['FLASHINFER', 'TRIT0
INFO 07-17 12:25:46 [bitsandbytes_loader.py:786] Loading weights with BitsAndBytes quantization. May take a while ...
model.safetensors: 100%      1.53G/1.53G [00:10<00:00, 217MB/s]
INFO 07-17 12:25:58 [weight_utils.py:581] Time spent downloading weights for unsloth/qwen2.5-1.5b-instruct-unsloth-bn
INFO 07-17 12:25:58 [weight_utils.py:625] No model.safetensors.index.json found in remote.

Loading safetensors checkpoint shards: 100% Completed | 1/1 [00:00<00:00, 8.90it/s]

Loading safetensors checkpoint shards: 100% Completed | 1/1 [00:01<00:00, 1.66s/it]
INFO 07-17 12:26:00 [punica_selector.py:20] Using PunicaWrapperGPU.
INFO 07-17 12:26:01 [gpu_model_runner.py:4820] Model loading took 1.46 GiB memory and 14.207626 seconds
INFO 07-17 12:26:18 [backends.py:1051] Using cache directory: /root/.cache/vllm/torch_compile_cache/9c211855b9/rank_0
INFO 07-17 12:26:18 [backends.py:1111] Dynamo bytecode transform time: 15.62 s
Unsloth: Compiling kernels: 0it [00:00, ?it/s] INFO 07-17 12:26:26 [backends.py:372] Cache the graph of compile range

Unsloth: Compiling kernels: 100% [██████████] 6/6 [00:01<00:00, 3.06it/s, triton_poi_fused_add_cat_index_select_mul_s
Unsloth: Compiling kernels: 100% [██████████] 6/6 [00:00<00:00, 294.32it/s, triton_poi_fused_add_cat_index_select_mul_s
Unsloth: Compiling kernels: 100% [██████████] 6/6 [00:00<00:00, 249.86it/s, triton_poi_fused_add_cat_index_select_mul_s
Unsloth: Compiling kernels: 100% [██████████] 5/5 [00:00<00:00, 9.54it/s, triton_poi_fused_add_cat_index_select_mul_s
Unsloth: Compiling kernels: 100% [██████████] 3/3 [00:00<00:00, 15.82it/s, triton_red_fused__to_copy_add_mean_mul_pow_
INFO 07-17 12:26:41 [backends.py:390] Compiling a graph for compile range (1, 4096) takes 22.90 s
INFO 07-17 12:26:45 [decorators.py:655] saved AOT compiled function to /root/.cache/vllm/torch_compile_cache/torch_aot
INFO 07-17 12:26:45 [monitor.py:48] torch.compile took 43.29 s in total
INFO 07-17 12:26:47 [monitor.py:76] Initial profiling/warmup run took 1.31 s
INFO 07-17 12:26:58 [kv_cache_utils.py:829] Overriding num_gpu_blocks=0 with num_gpu_blocks_override=64
INFO 07-17 12:26:58 [gpu_model_runner.py:5876] Profiling CUDA graph memory: PIECEWISE=22 (largest=64), FULL=14 (large
WARNING 07-17 12:26:59 [utils.py:267] Using default LoRA kernel configs
INFO 07-17 12:29:53 [gpu_model_runner.py:5955] Estimated CUDA graph memory: 0.75 GiB total
INFO 07-17 12:29:54 [gpu_worker.py:436] Available KV cache memory: 5.99 GiB
INFO 07-17 12:29:54 [gpu_worker.py:470] In v0.19, CUDA graph memory profiling will be enabled by default (VLLM_MEMORY
INFO 07-17 12:29:54 [kv_cache_utils.py:1319] GPU KV cache size: 224,192 tokens
INFO 07-17 12:29:54 [kv_cache_utils.py:1324] Maximum concurrency for 1,024 tokens per request: 218.94x
INFO 07-17 12:29:54 [kernel_warmup.py:69] Warming up FlashInfer attention.
INFO 07-17 12:29:55 [vllm_utils.py:744] Unsloth: Running patched vLLM v1 `capture_model`.
Capturing CUDA graphs (mixed prefill-decode, PIECEWISE): 100% [██████████] 22/22 [00:33<00:00, 1.51s/it]
Capturing CUDA graphs (decode, FULL): 100% [██████████] 14/14 [00:03<00:00, 4.37it/s] INFO 07-17 12:30:31 [gpu_model_r
INFO 07-17 12:30:31 [vllm_utils.py:751] Unsloth: Patched vLLM v1 graph capture finished in 36 secs.

```



```
INFO 07-17 12:30:33 [gpu_worker.py:597] CUDA graph pool memory: 0.46 GiB (actual), 0.75 GiB (estimated), difference:
INFO 07-17 12:30:33 [core.py:283] init engine (profile, create kv cache, warmup model) took 271.92 seconds
`torch_dtype` is deprecated! Use `dtype` instead!
Unsloth: Just some info: will skip parsing ['attention_norm', 'layer_norm2', 'post_feedforward_layernorm', 'post_per_
Some weights of Qwen2ForCausalLM were not initialized from the model checkpoint at unsloth/qwen2.5-1.5b-instruct-uns
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
Performing substitution for additional_keys=set()
Unsloth: Just some info: will skip parsing ['attention_norm', 'layer_norm2', 'post_feedforward_layernorm', 'post_per_
Unsloth 2026.7.3 patched 28 layers with 28 QKV layers, 28 O layers and 28 MLP layers.

README.md:      4.02k/? [00:00<00:00, 170kB/s]

derived_procedure_train.tsv:      1.13M/? [00:00<00:00, 14.2MB/s]

expert_train.tsv:      128k/? [00:00<00:00, 5.49MB/s]

procedure_train.tsv:      874k/? [00:00<00:00, 17.9MB/s]

tram_train.tsv:      546k/? [00:00<00:00, 13.8MB/s]

derived_procedure_dev.tsv:      215k/? [00:00<00:00, 11.1MB/s]

expert_dev.tsv:      19.0k/? [00:00<00:00, 1.48MB/s]

procedure_dev.tsv:      153k/? [00:00<00:00, 9.89MB/s]

tram_dev.tsv:      97.4k/? [00:00<00:00, 6.19MB/s]

derived_procedure_test.tsv:      226k/? [00:00<00:00, 13.3MB/s]

expert_test.tsv:      79.7k/? [00:00<00:00, 5.78MB/s]

procedure_test.tsv:      184k/? [00:00<00:00, 10.7MB/s]

tram_test.tsv:      116k/? [00:00<00:00, 7.63MB/s]

Generating train split: 100%                                14936/14936 [00:00<00:00, 14291.44 examples/s]

Generating validation split: 100%                          2630/2630 [00:00<00:00, 52273.77 examples/s]

Generating test split: 100%                                3170/3170 [00:00<00:00, 62447.48 examples/s]

Map: 100%                                                   14936/14936 [00:01<00:00, 14839.96 examples/s]

Filter: 100%                                                14936/14936 [00:00<00:00, 50460.11 examples/s]

[data] loaded Security-TTP-Mapping: 14936 rows
==(=====)== Unsloth - 2x faster free finetuning | Num GPUs used = 1
  \ \ / | Num examples = 14,936 | Num Epochs = 1 | Total steps = 250
0^0/ \_/ \ Batch size per device = 4 | Gradient accumulation steps = 1
 \ ____/ / Data Parallel GPUs = 1 | Total batch size (4 x 1 x 1) = 4
  "-__-_" Trainable parameters = 18,464,768 of 1,562,179,072 (1.18% trained)
WARNING 07-17 12:30:52 [input_processor.py:149] vLLM has deprecated support for supporting different tokenizers for d
Unsloth: Will smartly offload gradients to save VRAM!
[250/250 38:27, Epoch 0/1]
```

Step	Training Loss	reward	reward_std	completions / mean_length	completions / min_length	completions / max_length	completions / clipped_ratio	completions / mean_terminated_length	comp min
1	0.197000	-0.375000	0.629153	119.750000	72.000000	195.000000	0.000000	119.750000	
2	-0.000000	0.500000	0.000000	104.750000	71.000000	170.000000	0.000000	104.750000	
3	-0.056500	-0.875000	0.250000	230.000000	165.000000	256.000000	0.500000	204.000000	
4	-0.204000	0.875000	1.887459	93.250000	53.000000	136.000000	0.000000	93.250000	
5	0.174100	-0.125000	0.750000	101.250000	37.000000	124.000000	0.000000	101.250000	
6	0.000000	0.500000	0.000000	109.750000	91.000000	151.000000	0.000000	109.750000	
7	0.020500	-0.125000	0.750000	73.000000	46.000000	95.000000	0.000000	73.000000	
8	0.078300	0.125000	0.750000	124.500000	45.000000	202.000000	0.000000	124.500000	
9	0.009400	-0.500000	0.707107	112.750000	45.000000	256.000000	0.250000	65.000000	
10	0.000000	0.500000	0.000000	107.750000	72.000000	158.000000	0.000000	107.750000	
11	0.464700	-0.125000	0.721688	129.500000	37.000000	256.000000	0.250000	87.333336	
12	0.397400	0.125000	0.750000	97.500000	62.000000	175.000000	0.000000	97.500000	
13	0.065700	0.250000	0.500000	112.250000	72.000000	148.000000	0.000000	112.250000	
14	0.000000	0.500000	0.000000	87.750000	63.000000	115.000000	0.000000	87.750000	
15	0.492100	0.250000	0.500000	129.000000	68.000000	256.000000	0.250000	86.666672	
16	0.216600	-0.375000	0.629153	221.500000	118.000000	256.000000	0.750000	118.000000	

17	0.487800	-0.125000	0.721688	163.750000	66.000000	256.000000	0.500000	71.500000
18	0.078700	0.187500	0.625000	108.000000	45.000000	134.000000	0.000000	108.000000
19	0.263000	-0.125000	0.750000	102.000000	67.000000	165.000000	0.000000	102.000000
20	0.326800	-0.187500	0.800391	127.750000	47.000000	256.000000	0.250000	85.000000
21	0.126700	0.187500	0.625000	129.250000	81.000000	174.000000	0.000000	129.250000
22	-0.040200	0.250000	0.500000	174.000000	121.000000	229.000000	0.000000	174.000000
23	-0.103300	0.250000	0.500000	119.750000	88.000000	170.000000	0.000000	119.750000
24	0.133800	0.125000	0.750000	116.750000	72.000000	148.000000	0.000000	116.750000
25	0.353000	-0.437500	0.657489	169.500000	77.000000	256.000000	0.500000	83.000000
26	0.132400	-0.187500	0.800391	96.250000	64.000000	127.000000	0.000000	96.250000
27	0.263600	-0.250000	0.866025	196.250000	88.000000	256.000000	0.500000	136.500000
28	0.631100	0.187500	0.625000	85.750000	39.000000	194.000000	0.000000	85.750000
29	0.000000	0.500000	0.000000	94.250000	68.000000	116.000000	0.000000	94.250000
30	0.010300	0.250000	0.500000	145.000000	86.000000	250.000000	0.000000	145.000000
31	0.000000	0.500000	0.000000	121.250000	65.000000	184.000000	0.000000	121.250000
32	0.000000	0.500000	0.000000	139.250000	59.000000	219.000000	0.000000	139.250000
33	0.008300	0.250000	0.500000	105.250000	91.000000	127.000000	0.000000	105.250000
34	-0.143700	0.750000	1.258306	94.000000	76.000000	126.000000	0.000000	94.000000
35	0.000000	0.500000	0.000000	120.750000	78.000000	143.000000	0.000000	120.750000
36	-0.250000	0.187500	0.625000	76.000000	38.000000	98.000000	0.000000	76.000000
37	-0.162500	0.187500	0.625000	100.750000	68.000000	140.000000	0.000000	100.750000
38	0.000000	0.500000	0.000000	105.250000	83.000000	123.000000	0.000000	105.250000
39	0.000000	0.500000	0.000000	126.250000	85.000000	171.000000	0.000000	126.250000
40	-0.046400	0.250000	0.500000	158.750000	101.000000	215.000000	0.000000	158.750000
41	0.000000	0.500000	0.000000	108.750000	90.000000	134.000000	0.000000	108.750000
42	0.000000	0.500000	0.000000	97.500000	45.000000	200.000000	0.000000	97.500000
43	0.250000	0.250000	0.500000	74.000000	35.000000	111.000000	0.000000	74.000000
44	0.163400	-0.250000	0.500000	104.000000	65.000000	142.000000	0.000000	104.000000
45	0.000000	0.500000	0.000000	112.000000	75.000000	183.000000	0.000000	112.000000
46	0.239300	0.250000	0.500000	94.000000	66.000000	139.000000	0.000000	94.000000
47	0.000000	0.500000	0.000000	62.250000	36.000000	91.000000	0.000000	62.250000
48	0.538500	0.125000	0.750000	123.250000	65.000000	256.000000	0.250000	79.000000
49	-0.156700	0.125000	0.750000	100.500000	61.000000	200.000000	0.000000	100.500000
50	-0.139100	0.250000	0.500000	97.000000	70.000000	122.000000	0.000000	97.000000
51	-0.145800	0.250000	0.500000	96.000000	51.000000	190.000000	0.000000	96.000000
52	0.000000	0.500000	0.000000	92.000000	69.000000	110.000000	0.000000	92.000000
53	0.000000	0.500000	0.000000	101.750000	73.000000	134.000000	0.000000	101.750000
54	0.000000	0.500000	0.000000	134.250000	35.000000	232.000000	0.000000	134.250000
55	0.000000	0.500000	0.000000	113.250000	63.000000	163.000000	0.000000	113.250000
56	0.422500	0.000000	0.577350	72.250000	26.000000	152.000000	0.000000	72.250000
57	0.000000	0.500000	0.000000	132.250000	89.000000	184.000000	0.000000	132.250000
58	0.164400	0.250000	0.500000	74.500000	30.000000	99.000000	0.000000	74.500000
59	0.177000	0.000000	0.577350	113.750000	79.000000	173.000000	0.000000	113.750000
60	0.000000	0.500000	0.000000	120.500000	45.000000	197.000000	0.000000	120.500000
61	0.000000	0.500000	0.000000	81.500000	60.000000	106.000000	0.000000	81.500000
62	0.000000	0.500000	0.000000	63.000000	49.000000	75.000000	0.000000	63.000000
63	0.000000	0.500000	0.000000	113.250000	75.000000	152.000000	0.000000	113.250000

64	0.000000	0.500000	0.000000	127.500000	81.000000	184.000000	0.000000	127.500000
65	0.000000	0.500000	0.000000	72.750000	36.000000	102.000000	0.000000	72.750000
66	0.009600	0.187500	0.625000	104.000000	75.000000	139.000000	0.000000	104.000000
67	0.000000	0.500000	0.000000	107.500000	47.000000	161.000000	0.000000	107.500000
68	0.000000	0.500000	0.000000	100.250000	47.000000	214.000000	0.000000	100.250000
69	-0.214200	0.250000	0.500000	78.750000	45.000000	98.000000	0.000000	78.750000
70	0.507800	0.250000	0.500000	127.000000	34.000000	256.000000	0.250000	84.000000
71	0.000000	0.500000	0.000000	86.000000	55.000000	153.000000	0.000000	86.000000
72	0.000000	0.500000	0.000000	54.000000	34.000000	93.000000	0.000000	54.000000
73	-0.200600	0.250000	0.500000	83.500000	50.000000	102.000000	0.000000	83.500000
74	0.000000	0.500000	0.000000	82.250000	61.000000	107.000000	0.000000	82.250000
75	0.000000	0.500000	0.000000	96.750000	64.000000	122.000000	0.000000	96.750000
76	0.000000	0.500000	0.000000	110.000000	85.000000	135.000000	0.000000	110.000000
77	0.000000	0.500000	0.000000	72.000000	37.000000	99.000000	0.000000	72.000000
78	0.061700	0.000000	0.577350	87.750000	71.000000	101.000000	0.000000	87.750000
79	0.000000	0.500000	0.000000	104.500000	68.000000	150.000000	0.000000	104.500000
80	0.253800	-0.062500	0.657489	130.250000	60.000000	256.000000	0.250000	88.333336
81	0.000000	0.500000	0.000000	111.500000	86.000000	146.000000	0.000000	111.500000
82	0.000000	0.500000	0.000000	77.000000	33.000000	99.000000	0.000000	77.000000
83	0.000000	0.500000	0.000000	103.250000	58.000000	152.000000	0.000000	103.250000
84	0.269600	0.250000	0.500000	92.250000	48.000000	142.000000	0.000000	92.250000
85	0.488300	0.250000	0.500000	129.500000	51.000000	256.000000	0.250000	87.333336
86	0.000000	0.500000	0.000000	116.750000	85.000000	175.000000	0.000000	116.750000
87	0.000000	0.500000	0.000000	122.500000	54.000000	221.000000	0.000000	122.500000
88	0.000000	0.500000	0.000000	103.750000	52.000000	179.000000	0.000000	103.750000
89	-0.109700	0.250000	0.500000	92.250000	72.000000	123.000000	0.000000	92.250000
90	0.789600	0.125000	0.750000	99.250000	30.000000	256.000000	0.250000	47.000000
91	0.000000	0.500000	0.000000	126.750000	86.000000	213.000000	0.000000	126.750000
92	0.000000	0.500000	0.000000	64.500000	30.000000	127.000000	0.000000	64.500000
93	0.000000	0.500000	0.000000	133.750000	59.000000	238.000000	0.000000	133.750000
94	0.000000	0.500000	0.000000	111.250000	74.000000	147.000000	0.000000	111.250000
95	0.000000	0.500000	0.000000	94.000000	61.000000	126.000000	0.000000	94.000000
96	-0.123600	1.250000	1.500000	65.750000	36.000000	82.000000	0.000000	65.750000
97	0.000000	0.500000	0.000000	84.000000	76.000000	93.000000	0.000000	84.000000
98	0.000000	0.500000	0.000000	122.250000	87.000000	144.000000	0.000000	122.250000
99	0.000000	0.500000	0.000000	65.500000	50.000000	84.000000	0.000000	65.500000
100	0.000000	0.500000	0.000000	102.500000	59.000000	144.000000	0.000000	102.500000
101	0.000000	0.500000	0.000000	115.500000	68.000000	189.000000	0.000000	115.500000
102	0.000000	0.500000	0.000000	94.500000	87.000000	107.000000	0.000000	94.500000
103	0.000000	0.500000	0.000000	112.000000	86.000000	145.000000	0.000000	112.000000
104	-0.146700	0.250000	0.500000	92.000000	65.000000	118.000000	0.000000	92.000000
105	0.663500	0.187500	0.625000	110.000000	31.000000	256.000000	0.250000	61.333336
106	0.000000	0.500000	0.000000	84.500000	72.000000	115.000000	0.000000	84.500000
107	0.000000	0.500000	0.000000	91.000000	67.000000	119.000000	0.000000	91.000000
108	0.000000	0.500000	0.000000	45.000000	32.000000	64.000000	0.000000	45.000000
109	0.000000	0.500000	0.000000	96.500000	52.000000	192.000000	0.000000	96.500000
110	0.000000	0.500000	0.000000	77.250000	58.000000	92.000000	0.000000	77.250000

111	0.337900	0.250000	0.500000	152.750000	83.000000	256.000000	0.250000	118.333336
112	0.132300	0.250000	0.500000	68.000000	39.000000	86.000000	0.000000	68.000000
113	0.000000	0.500000	0.000000	92.750000	59.000000	149.000000	0.000000	92.750000
114	0.000000	0.500000	0.000000	74.750000	63.000000	81.000000	0.000000	74.750000
115	0.000000	0.500000	0.000000	81.000000	61.000000	104.000000	0.000000	81.000000
116	0.139800	0.250000	0.500000	99.250000	65.000000	127.000000	0.000000	99.250000
117	0.000000	0.500000	0.000000	77.250000	72.000000	82.000000	0.000000	77.250000
118	0.460500	0.125000	0.750000	133.250000	61.000000	256.000000	0.250000	92.333336
119	0.306300	0.125000	0.750000	158.750000	69.000000	256.000000	0.250000	126.333336
120	0.000000	0.500000	0.000000	64.500000	50.000000	83.000000	0.000000	64.500000
121	0.000000	0.500000	0.000000	104.750000	76.000000	128.000000	0.000000	104.750000
122	0.000000	0.500000	0.000000	112.500000	74.000000	204.000000	0.000000	112.500000
123	0.528000	0.125000	0.750000	124.500000	32.000000	256.000000	0.250000	80.666672
124	0.000000	0.500000	0.000000	71.750000	40.000000	100.000000	0.000000	71.750000
125	0.000000	0.500000	0.000000	154.250000	129.000000	199.000000	0.000000	154.250000
126	0.000000	0.500000	0.000000	70.000000	47.000000	101.000000	0.000000	70.000000
127	0.000000	0.500000	0.000000	128.250000	81.000000	208.000000	0.000000	128.250000
128	0.000000	0.500000	0.000000	90.500000	52.000000	115.000000	0.000000	90.500000
129	0.119100	0.125000	0.750000	206.750000	127.000000	256.000000	0.250000	190.333344
130	0.000000	0.500000	0.000000	113.250000	68.000000	154.000000	0.000000	113.250000
131	0.000000	0.500000	0.000000	88.750000	78.000000	113.000000	0.000000	88.750000
132	0.460500	0.125000	0.750000	133.250000	63.000000	256.000000	0.250000	92.333336
133	0.000000	0.500000	0.000000	73.000000	61.000000	99.000000	0.000000	73.000000
134	-0.137600	2.750000	1.500000	111.750000	66.000000	174.000000	0.000000	111.750000
135	-0.121400	0.187500	0.625000	121.500000	92.000000	174.000000	0.000000	121.500000
136	0.000000	0.500000	0.000000	110.250000	55.000000	132.000000	0.000000	110.250000
137	0.057800	0.250000	0.500000	125.500000	75.000000	202.000000	0.000000	125.500000
138	0.104200	0.187500	0.625000	58.750000	41.000000	71.000000	0.000000	58.750000
139	0.000000	0.500000	0.000000	80.750000	38.000000	132.000000	0.000000	80.750000
140	0.233500	0.125000	0.750000	174.500000	82.000000	256.000000	0.250000	147.333344
141	0.000000	0.500000	0.000000	128.250000	99.000000	158.000000	0.000000	128.250000
142	0.000000	0.500000	0.000000	92.000000	58.000000	124.000000	0.000000	92.000000
143	0.000000	0.500000	0.000000	98.750000	47.000000	150.000000	0.000000	98.750000
144	0.000000	0.500000	0.000000	111.750000	88.000000	135.000000	0.000000	111.750000
145	0.000000	0.500000	0.000000	103.500000	34.000000	189.000000	0.000000	103.500000
146	0.000000	0.500000	0.000000	74.000000	35.000000	139.000000	0.000000	74.000000
147	0.000000	0.500000	0.000000	135.750000	105.000000	160.000000	0.000000	135.750000
148	0.584600	0.250000	0.500000	118.000000	31.000000	256.000000	0.250000	72.000000
149	0.000000	0.500000	0.000000	77.250000	45.000000	100.000000	0.000000	77.250000
150	0.000000	0.500000	0.000000	106.500000	77.000000	131.000000	0.000000	106.500000
151	0.000000	0.500000	0.000000	139.500000	55.000000	195.000000	0.000000	139.500000
152	0.000000	0.500000	0.000000	57.750000	34.000000	95.000000	0.000000	57.750000
153	0.000000	0.500000	0.000000	66.750000	29.000000	96.000000	0.000000	66.750000
154	0.000000	0.500000	0.000000	87.000000	54.000000	132.000000	0.000000	87.000000
155	0.000000	0.500000	0.000000	73.250000	49.000000	91.000000	0.000000	73.250000
156	0.000000	0.500000	0.000000	69.250000	34.000000	84.000000	0.000000	69.250000
157	-0.189400	0.250000	0.500000	149.750000	92.000000	247.000000	0.000000	149.750000

158	0.401400	0.125000	0.750000	142.000000	90.000000	256.000000	0.250000	104.000000
159	0.000000	0.500000	0.000000	54.250000	42.000000	63.000000	0.000000	54.250000
160	0.000000	0.500000	0.000000	81.000000	64.000000	96.000000	0.000000	81.000000
161	0.000000	0.500000	0.000000	100.500000	71.000000	142.000000	0.000000	100.500000
162	0.000000	0.500000	0.000000	68.750000	48.000000	87.000000	0.000000	68.750000
163	0.000000	0.500000	0.000000	93.000000	30.000000	178.000000	0.000000	93.000000
164	0.000000	0.500000	0.000000	64.500000	37.000000	89.000000	0.000000	64.500000
165	0.427500	0.125000	0.750000	138.000000	88.000000	256.000000	0.250000	98.666672
166	0.000000	0.500000	0.000000	50.000000	31.000000	70.000000	0.000000	50.000000
167	0.000000	0.500000	0.000000	81.500000	66.000000	88.000000	0.000000	81.500000
168	0.000000	0.500000	0.000000	113.500000	55.000000	179.000000	0.000000	113.500000
169	0.000000	0.500000	0.000000	130.000000	80.000000	184.000000	0.000000	130.000000
170	-0.438500	0.125000	0.750000	57.000000	7.000000	84.000000	0.000000	57.000000
171	0.000000	0.500000	0.000000	58.750000	40.000000	71.000000	0.000000	58.750000
172	0.000000	0.500000	0.000000	98.250000	70.000000	166.000000	0.000000	98.250000
173	0.000000	0.500000	0.000000	89.250000	65.000000	119.000000	0.000000	89.250000
174	0.000000	0.500000	0.000000	86.750000	69.000000	117.000000	0.000000	86.750000
175	0.000000	0.500000	0.000000	75.750000	35.000000	111.000000	0.000000	75.750000
176	0.000000	0.500000	0.000000	74.750000	48.000000	109.000000	0.000000	74.750000
177	0.000000	0.500000	0.000000	96.000000	60.000000	139.000000	0.000000	96.000000
178	0.000000	0.500000	0.000000	152.500000	85.000000	211.000000	0.000000	152.500000
179	0.000000	0.500000	0.000000	92.000000	37.000000	136.000000	0.000000	92.000000
180	0.015600	0.250000	0.500000	88.250000	70.000000	118.000000	0.000000	88.250000
181	0.343900	-0.187500	0.800391	144.500000	68.000000	256.000000	0.250000	107.333336
182	0.000000	0.500000	0.000000	65.000000	44.000000	86.000000	0.000000	65.000000
183	0.000000	0.500000	0.000000	63.000000	51.000000	80.000000	0.000000	63.000000
184	0.000000	0.500000	0.000000	109.000000	65.000000	143.000000	0.000000	109.000000
185	0.000000	0.500000	0.000000	73.500000	63.000000	92.000000	0.000000	73.500000
186	0.000000	0.500000	0.000000	152.250000	69.000000	220.000000	0.000000	152.250000
187	0.000000	0.500000	0.000000	110.500000	68.000000	196.000000	0.000000	110.500000
188	0.186700	-0.062500	0.657489	75.250000	37.000000	105.000000	0.000000	75.250000
189	0.200500	0.250000	0.500000	103.500000	69.000000	145.000000	0.000000	103.500000
190	0.000000	0.500000	0.000000	76.500000	50.000000	92.000000	0.000000	76.500000
191	0.000000	0.500000	0.000000	73.500000	38.000000	114.000000	0.000000	73.500000
192	0.000000	0.500000	0.000000	124.500000	58.000000	172.000000	0.000000	124.500000
193	0.000000	0.500000	0.000000	90.500000	58.000000	163.000000	0.000000	90.500000
194	0.000000	0.500000	0.000000	118.000000	71.000000	188.000000	0.000000	118.000000
195	0.000000	0.500000	0.000000	94.000000	72.000000	150.000000	0.000000	94.000000
196	0.000000	0.500000	0.000000	66.000000	45.000000	112.000000	0.000000	66.000000
197	0.000000	0.500000	0.000000	100.000000	84.000000	117.000000	0.000000	100.000000
198	0.000000	0.500000	0.000000	72.500000	58.000000	90.000000	0.000000	72.500000
199	0.000000	0.500000	0.000000	72.000000	55.000000	85.000000	0.000000	72.000000
200	-0.094200	0.250000	0.500000	104.750000	78.000000	142.000000	0.000000	104.750000
201	0.000000	0.500000	0.000000	60.000000	31.000000	87.000000	0.000000	60.000000
202	-0.446400	0.125000	0.750000	65.250000	7.000000	113.000000	0.000000	65.250000
203	0.000000	0.500000	0.000000	105.000000	71.000000	126.000000	0.000000	105.000000
204	0.000000	0.500000	0.000000	81.750000	68.000000	93.000000	0.000000	81.750000

204	0.000000	0.500000	0.000000	81.500000	58.000000	102.000000	0.000000	81.500000
205	0.000000	0.500000	0.000000	79.500000	57.000000	102.000000	0.000000	79.500000
206	0.000000	0.500000	0.000000	82.000000	41.000000	117.000000	0.000000	82.000000
207	0.000000	0.000000	0.573050	110.500000	81.000000	850.000000	0.050000	80.000000